

Retrieval Basics: Lexical, Dense, and Hybrid Search

LUMINA gold corpus, document 1 of 4. FDE Agent Engineering Bootcamp. CC BY 4.0.

Why retrieval is a decision, not a step

A retrieval system answers one question: given a query, which passages should the model read before it writes? Everything else—the embedding model, the index type, the fusion formula—is machinery in service of that question. Two systems with identical models can differ by twenty points of recall because one of them chunked its documents well and the other did not.

The measurements in this document assume a fixed corpus and a fixed set of judged queries. Without both, a retrieval number is not comparable to anything, including its own value last week.

Lexical retrieval and BM25

Lexical retrieval matches the words in the query against the words in the document. The standard scoring function is BM25, a probabilistic ranking function that refines TF-IDF with two ideas: term frequency saturates, and long documents are penalized.

BM25 scores a document D for a query Q as a sum over query terms of the inverse document frequency of the term, multiplied by a saturating term-frequency factor. That factor is governed by two free parameters:

- " k_1 controls how quickly term frequency saturates. The common default is 1.2. Higher values let repeated terms keep adding score for longer.

- " b controls length normalization, on a scale from 0 (no normalization) to 1 (full). The common default is 0.75.

BM25's strengths are the reasons it has not been retired: it needs no training, it is fast, it is interpretable, and it handles rare exact tokens—a product SKU, an error code, a surname—that dense models routinely miss. Its weakness is vocabulary mismatch. A query for "car" does not match a document that only says "automobile", because BM25 has no notion that the two words are related.

Dense retrieval and embeddings

Dense retrieval maps text into a vector space where semantic similarity is geometric proximity. An embedding model turns a passage into a fixed-length vector; the same model turns the query into a vector; the search returns the passages whose vectors are nearest.

Nearness is usually cosine similarity, which measures the angle between two vectors and

ignores their magnitude. For unit-normalized vectors, cosine similarity and the dot product give the same ranking, which is why many vector databases accept either.

Vector length is a property of the model, not a tuning choice. OpenAI's text-embedding-3-small produces vectors of 1536 dimensions, and every vector in one index must have the same dimensionality mixing models in a single index is a configuration error, not a trade-off.

Dense retrieval fixes the vocabulary mismatch that BM25 cannot: "how do I stop my page from loading slowly" can retrieve a passage about latency budgets that shares almost no words with the query. It fails in the mirror image of BM25's failure: on exact rare tokens, where the nearest neighbours of a novel string are semantically plausible and factually wrong.

Hybrid retrieval

Because the two methods fail differently, running both and combining them beats either. This is hybrid retrieval, and the combination step is where implementations diverge.

Score-based fusion normalize each system's scores and add them is tempting and fragile. BM25 scores are unbounded and corpus-dependent; cosine similarities sit in a narrow band near the top of the ranking. Normalizing across that mismatch means picking a scaling constant that has no principled value and drifts as the corpus grows.

Reciprocal rank fusion (RRF) avoids the problem by discarding the scores and keeping only the ranks. For a document d appearing in result lists $L_1 \dots L_n$, its fused score is:

$$\text{RRF}(d) = \sum \text{over } i \text{ of } 1 / (k + \text{rank}_i(d))$$

where $\text{rank}_i(d)$ is the document's 1-based position in list i , and documents absent from a list contribute nothing. The constant k damps the influence of top ranks; the value from the original 2009 paper by Cormack, Clarke, and Buettcher is 60, and it is the default in most implementations because it works acceptably almost everywhere.

RRF's appeal is that it needs no score calibration, no training data, and no per-corpus tuning, and it is stable when one retriever returns garbage: a bad list contributes small reciprocals and gets outvoted. Its cost is that it throws away real information a retriever that is confident and a retriever that is guessing look the same once you keep only ranks.

Chunking

Retrieval returns chunks, not documents, so chunk boundaries decide what the model can see. Three failure modes recur:

" Chunks too small. A 100-token chunk retrieves precisely and then hands the model a fragment with no context. The answer is right about the fragment and wrong about the document.

" Chunks too large. A 4000-token chunk almost always contains the answer somewhere, which makes recall look excellent and precision meaningless. The model reads mostly irrelevant text and cites a chunk that is technically correct and useless to a reader.

" Chunks that ignore structure. Splitting every 500 tokens regardless of headings severs a table from its caption and a definition from its term.

Practical defaults for prose: 300 to 800 tokens per chunk with **10 to 20 percent overlap**, split on structural boundaries headings first, then paragraphs, then sentences and never mid-sentence. Overlap exists so that a fact sitting on a boundary appears whole in at least one chunk.

Every chunk must carry a locator back to its source: a page number for a PDF, a heading or line number for text. A citation without a locator cannot be verified by a reader, and a citation a reader cannot verify is decoration.

Top-k, thresholds, and re-ranking

k is how many chunks you retrieve. Small k starves the model; large k buries the relevant chunk in noise and costs tokens on every request. Retrieve wider than you intend to use 20 to 50 candidates then narrow.

Narrowing is what a re-ranker does. A cross-encoder scores each (query, passage) pair jointly rather than comparing two independently-computed vectors, which is far more accurate and far more expensive: cost is linear in the number of candidates, so it is only affordable on a shortlist. A typical production shape is: retrieve 50 by hybrid search, re-rank, keep the top 5.

A similarity threshold is the other half. Without one, a query with no relevant documents still returns k results, and the model dutifully answers from the best of a bad set. With one, empty retrieval is possible and an honest system says it found nothing rather than citing its way to a wrong answer.

Measuring it

Retrieval metrics are computed over a judged set of queries, each with known relevant passages. Two that matter here:

" Recall@k the share of queries for which at least one relevant passage appears in the top k. It answers: did the model get a chance to be right?

" Precision@k the share of the top k that is relevant. It answers: how much of what we paid for was useful?

Recall@5 is the standard gate for a RAG system because 5 chunks is roughly what fits in a prompt alongside instructions and history. A recall@5 of 0.70 means three in ten questions are unanswerable no matter how good the model is the evidence never arrived.

Judged sets need to be big enough to mean something. Below about thirty queries, a single question flipping moves the score by more than three points, and the number is an anecdote with a decimal point.